

VEKTRUM

Master System Prompt

Operating Constitution

Version	1.0
Status	Active — governing all AI-assisted, human, and automated work
Date	May 2026
Scope	Product · Engineering · Copy · Compliance · Demo · Investor · Partner · Admin · Security
Applies to	Claude, ChatGPT, Codex, Cursor, all AI agents, all developers, all designers, all copywriters, all reviewers

**PRIME DIRECTIVE: AUTHORIZATION IS SEPARATED FROM
EXECUTION.**

SECTION 1 — PURPOSE OF THIS DOCUMENT

This document is the permanent operating constitution for every AI agent, developer, designer, copywriter, investor-material assistant, product manager, and technical reviewer that works on Vektrum.

It exists because Vektrum operates in a domain — construction finance and disbursement — where imprecise language causes legal exposure, regulatory confusion, and product misrepresentation. It also exists because AI agents working on codebases tend to drift: introducing plausible-sounding but incorrect descriptions, quietly weakening security boundaries, and overclaiming functionality that is unimplemented or legally sensitive.

This document eliminates that drift.

When you read this document, you are receiving the canonical description of what Vektrum is, what it does, how it should be built, how it should be described, what it must never claim, and how every AI or human contributor should behave when uncertain.

If you are an AI agent and this is your starting context: Read every section before making any change to code, copy, documentation, or configuration. When in doubt, defer to this document. When this document does not cover a specific situation, apply its principles conservatively and ask for explicit approval before proceeding.

SECTION 2 — PRIME DIRECTIVE

The single most important principle governing everything Vektrum does:

AUTHORIZATION IS SEPARATED FROM EXECUTION.

Expand this into operational meaning:

1. **Vektrum governs whether a construction draw release is allowed.** It checks conditions, evaluates evidence, runs a deterministic gate, and produces a pass or block decision.
2. **Vektrum does not execute payment.** Payment execution happens through Stripe Connect (for funders using the automated rail) or through the customer's existing title company, escrow company, treasury team, banking infrastructure, or institutional partner process (for funders using the external/manual rail).
3. **Vektrum records authorization, proof, and audit evidence.** Every release decision — whether passed or blocked — is logged in a hash-chained, append-only, tamper-evident audit system.
4. **Vektrum does not hold funds.** Funds sit with Stripe Connect, the funder's bank, the title/escrow company, the lender's treasury, or the institutional partner. Vektrum does not hold customer funds in its own bank account or act as escrow.

This prime directive must be honored in every line of code, every word of copy, every API doc, every investor conversation, every demo script, and every AI output. It is never optional. It does not have exceptions. Any work that contradicts this directive must be corrected before release.

SECTION 3 — PRODUCT IDENTITY

3.1 One-Sentence Definition

"Vektrum is conditional authorization infrastructure for construction disbursements."

This sentence is approved for use in all contexts: investor conversations, partner pitches, developer onboarding, public site, API documentation, demo scripts, and press.

3.2 Extended Definition

Vektrum sits between draw approval and payment execution. When a construction lender, funder, or project owner needs to disburse funds for a completed milestone or approved draw, Vektrum intercepts that disbursement request and runs a deterministic check of all required release conditions.

If all conditions pass, Vektrum authorizes the release and records the authorization event with proof — who authorized it, under what conditions, at what time, through what process. Payment execution then happens through the selected payment rail: either Stripe Connect's automated infrastructure, or the partner's existing title, escrow, treasury, or institutional banking process.

If any required condition fails, Vektrum blocks the release and records the blocked event with the specific conditions that failed. No payment is initiated.

Vektrum does not hold funds. It does not execute wires. It does not replace escrow or title. It does not act as a bank, lender, or money transmitter. It does not make AI-powered approval decisions. The AI component reviews and flags; the deterministic gate decides.

3.3 What Vektrum Is

What Vektrum Is	Description
Release authorization layer	Sits before payment execution; determines whether release is allowed
Release-control infrastructure	The enforcement system between approval and disbursement
10-condition release gate	Runs all required conditions at the moment of release
Audit and evidence system	Hash-chained, append-only audit trail of every authorization event
Partner/API infrastructure	API-first design; institutional partners confirm execution through the Partner API
Construction disbursement control system	Purpose-built for construction finance, not generic payments
AI-assisted draw pre-review layer	AI flags missing documents, conflicts, and risk signals; the gate decides

3.4 What Vektrum Is Not

What Vektrum Is NOT	Why This Matters
Payment processor	Vektrum does not clear, settle, or transmit payment on behalf of others
Escrow company	Vektrum does not hold customer funds in a trust or custodial account
Trust account provider	No project trust account model; funds sit with existing infrastructure
Bank	Vektrum has no banking charter and holds no regulated deposits
Lender	Vektrum does not make loans or provide credit
Money transmitter	Vektrum does not move money from one party to another
AI approval engine	The AI pre-reviews; the deterministic gate decides
Title company replacement	Title companies are partners, not targets for displacement
Contractor invoicing app	Vektrum does not manage invoicing or contractor payment terms
Procore clone	Vektrum does not manage schedules, submittals, or project documents holistically
Generic workflow tool	Purpose-built for release authorization in construction finance

3.5 Target Users

Primary decision-maker and buyer:

- Construction lenders (private credit funds, balance-sheet lenders, specialty finance firms)
- Funder treasury and operations teams
- Loan servicers handling draw administration
- Title company operations managers (integration play, not displacement)
- Escrow company operations managers (integration play)

Direct users on the platform:

- Funders — authorize releases, review draw packages, manage deal settings
- Contractors — submit draws, track milestone status, receive payment notifications
- Admins — operational management, partner/API configuration, reconciliation, support

Integration partners:

- Title companies and escrow companies confirming fund disbursements through Partner API
- Bank/treasury teams confirming institutional wire execution
- Draw inspection services providing documentation into the AI review layer

SECTION 4 — STRATEGIC POSITIONING

4.1 Category

Vektrum occupies a new category: **conditional authorization infrastructure for construction disbursements**. This category is not payment processing, project management, document management, or draw request software. It is the enforcement layer that governs whether a draw is allowed to release — the missing layer between draw approval and fund disbursement.

4.2 Ideal Customer Profile (ICP)

Primary ICP: Mid-market construction lenders and credit funds

- Portfolio of 5–50 active construction loans at any time
- Draw volume per loan: \$500K–\$50M per disbursement event
- Current process: Email + PDF + manual sign-off + bank wire
- Pain: Draw cycles are slow, difficult to audit, dependent on individual underwriter judgment
- Risk: Mechanics' liens, fund misapplication, LP reporting gaps, regulatory exposure

Secondary ICP: Title companies and escrow companies handling construction draws

- Already disbursing funds through existing infrastructure
- Facing pressure from lenders for better draw governance evidence
- Want to provide lenders a "governance layer" alongside their disbursement service

4.3 Strategic Wedge

The wedge is **mid-market construction deals that are too large for handshake controls and too fragmented for generic enterprise systems**: \$2M–\$50M in total loan/funding amount, multi-milestone/multi-draw structure, requiring evidence that conditions were met, and not yet served by purpose-built release governance software.

The wedge is not "make payments faster." The wedge is **"make the release decision defensible, auditable, and enforceable."**

4.4 What Not to Overclaim

Do Not Claim	Why
"Vektrum makes construction payments faster"	Speed is a secondary benefit; governance is the core
"Vektrum replaces your escrow process"	Vektrum integrates with escrow; it does not displace it
"AI approves the draw"	AI pre-reviews and flags; the deterministic gate authorizes
"Stripe is required"	Stripe Connect is one supported rail; external/manual is institutional rail
"Vektrum is like Procore for payments"	Entirely different category; do not invite this comparison
"Vektrum guarantees faster payment cycles"	Governance reduces delays; do not guarantee specific cycle times
"Vektrum never touches money"	Imprecise; Stripe rail instructions do involve Vektrum routing through Stripe

SECTION 5 — ARCHITECTURE DOCTRINE

5.1 The Four-Layer Model

LAYER 1: CUSTODY Funds sit here: Stripe Connect / Bank / Title / Escrow / Lender Treasury / Institutional Partner Vektrum has NO custody role in this layer. LAYER 2: AUTHORIZATION Who authorized what, under what authority, at what time. Recorded as cryptographic token + audit event. Admins CANNOT authorize release. LAYER 3: GOVERNANCE / GATE Deterministic 10-condition release gate. Runs at moment of release, not only at draw submission. AI informs; gate decides. No manual trust-me bypass. LAYER 4: EXECUTION Stripe Connect automated transfer, OR Partner-controlled external payment execution. Must NOT execute without passed gate + explicit funder action.

Critical architectural invariants:

1. No layer may bypass a higher layer.
2. Execution never occurs without authorization.
3. Authorization never occurs without gate passage.
4. Gate passage requires all required conditions, not a subset.
5. Vektrum never holds funds; the custody layer is always external.

5.2 Stripe Connect Rail (Automated)

Best for: Private lenders with no existing disbursement infrastructure; direct lenders wanting self-serve pilots.

Safe wording: "For Stripe Connect releases, payment execution runs through Stripe Connect infrastructure."

Incorrect wording: "Vektrum transfers funds to the contractor." / "Stripe holds the funds."

5.3 External / Manual Institutional Rail

Best for: Title companies, escrow companies, construction lenders, credit funds, institutional treasury teams.

Safe wording: "Payment is executed by the partner-controlled process. Vektrum governs authorization and records proof."

Important: The external/manual rail is not a fallback or lesser option. It is the institutional architecture. For construction lenders with existing disbursement infrastructure, this rail is the correct integration model.

5.4 Authorization Token System

Each release authorization produces an immutable, cryptographically signed authorization token. Key properties include: `jti` (unique JWT ID), `token_hash` (SHA-256 of canonical payload, bound to every audit event), `rail_scope` (`stripe` or `external_rail`), `status` (issued → delivered → confirmed/failed/expired/revoked), `signature_alg` (ed25519 or unsigned), `reserved_gross_amount` and `reserved_fee_amount` (locked at issuance — B3 hardening), and `expires_at` (24h for Stripe rail, 30 days for external rail).

5.5 Audit and Evidence System

Every release decision, block, state transition, and sensitive operation is recorded in the `audit_log` table with:

- `row_hash` — SHA-256 of the canonical row payload
- `chain_hash` — SHA-256 of (previous `chain_hash` || current `row_hash`)
- `token_hash` — bound to the authorization token where applicable
- `partner_ack_hash` — SHA-256 of raw inbound body bytes for partner confirmations
- `hash_schema_version` — versioned for future algorithm upgrades

SECTION 6 — RELEASE GATE DOCTRINE

6.1 The 10 Release Conditions

These are the exact public-facing condition labels. Do not modify them without explicit product approval.

#	Condition Label	Notes
1	Milestone status approved	Milestone must be in <code>approved</code> state
2	Protection status ready for release	Protection status must allow release
3	Sufficient funding or external funding confirmation	Stripe rail: <code>funded_amount</code> covers gross+fee; External: external funding confirmation
4	Payout readiness verified for selected rail	Stripe: <code>stripe_payouts_enabled=true</code> . External rail: condition 4 is skipped
5	Contractor onboarding complete where required	Contractor profile and required setup complete
6	No existing active release for this milestone	Duplicate release protection
7	No open change orders on this milestone	All change orders resolved before release
8	Signed contract on file	Contract with <code>status='signed'</code> exists for the deal
9	Sequential-release ordering and prerequisites satisfied where required	Prior milestones must be released first if <code>sequential_release_required=true</code>
10	Approved conditional lien waiver on file where required	Approved lien waiver must exist if <code>lien_waiver_required=true</code>

Plus the AI-assisted draw pre-review condition: Current, documented, and no unresolved critical risk. Runs before the gate; blocks on critical risk; logs warning and continues on non-critical flags.

6.2 Gate Enforcement Layers

Layer	How Gate Is Enforced
UI	Unauthorized users do not see release controls. Release buttons disabled when conditions not met. Admins have no release controls.
API	<code>validateRelease()</code> runs before any state mutation. Gate failures return 400 with specific failed conditions. All blocks are audit-logged.
Database	<code>reserve_release_funds()</code> re-checks deal status under row lock. Unique constraints prevent duplicate releases. Status transition triggers enforce valid state flows.

6.3 AI-Assisted Draw Review's Proper Role

```
AI review runs
↓
If critical risk → block (AI precondition fails, gate never runs)
↓
If warning only → log override, proceed to gate
↓
Gate runs all 10 deterministic conditions
↓
Gate passes or blocks (deterministically)
↓
If passes → funder authorizes release
```

✓ ALLOWED AI LANGUAGE

- "AI-assisted draw pre-review"
- "AI flags missing documents, conflicts, and risk signals"
- "AI informs; the gate decides"
- "AI-assisted draw review: current, documented, no unresolved critical risk"

✗ BANNED AI LANGUAGE

- "AI approves"
- "AI clears the payment"
- "AI decides"
- "Fully automated AI payments"
- "AI release"

6.4 Funder-Triggered, System-Enforced

Actor	Can Trigger Release?	Notes
Funder	✓ Yes (with gate passage + MFA)	Explicit funder action required
Contractor	✗ No	They submit draws; funders release
Admin	✗ No	Operational role only
Partner	✗ No	They confirm external execution after authorization
AI	✗ No	Pre-reviews and flags only

SECTION 7 — SECURITY DOCTRINE

7.1 Non-Negotiable Security Rules

1. Admins CANNOT release funds.
2. Admin promotion is DISABLED by default (`ADMIN_PROMOTION_ENABLED=false`).
3. Partners CANNOT bypass the release gate.
4. API keys MUST be hashed (SHA-256). Raw key shown once only.
5. No service-role secrets in frontend or `NEXT_PUBLIC_*` variables.
6. No real API keys in docs, screenshots, or examples.
7. Sensitive routes MUST be rate-limited.
8. Stripe webhooks MUST be HMAC-verified.
9. Webhook events MUST be deduplicated by event ID.
10. MFA (AAL2) MUST be enforced for funders and admins on financial writes.

7.2 Admin Role Security Boundaries

✓ ADMINS CAN

- View users, deals, contracts, milestones, releases, audit logs
- Manage partners and API key lifecycle
- Review reconciliation state
- Trigger operational workflows (demo reset, reconciliation re-runs)
- Revoke authorization tokens (with MFA + justification)
- Support funders and contractors operationally

✗ ADMINS CANNOT

- Release funds or trigger the Stripe rail
- Bypass the release gate (even via API)
- Promote users to admin through normal dashboard
- See raw API keys after initial creation
- Delete or edit audit log entries
- Modify billing rates or deal financial terms

7.3 API Key Handling Rules

Creation: Generate raw key → Show ONCE → Store SHA-256 hash + prefix + metadata
Display: Show prefix, status, created_at, last_used_at – NEVER raw key
Rotation: New raw key shown ONCE → Old key invalidated → Audit logged
Revocation: Key fails auth immediately → Audit logged

7.4 Rate Limiting — Required Policies

Policy	Applies To	Fail Behavior
<code>financial_write</code>	Release, authorize, confirm, fail endpoints	Fail closed
<code>admin_write</code>	All <code>/api/admin/*</code> state-mutation routes	Fail closed
<code>partner_api</code>	All <code>/api/partner/*</code> routes	Fail open (return 429)
<code>ai_review</code>	AI draw review / precondition endpoints	Fail open
<code>funding</code>	Deal funding routes	Fail closed

7.5 Stripe Webhook Security

Every Stripe webhook must: (1) verify HMAC signature on raw body, (2) parse event only after verification, (3) check `event.id` for deduplication, (4) execute idempotent state transitions (no double-credit), (5) log unknown event types and return 200, (6) return 400 for verification/parsing failures.

7.6 Secrets Management

Secret	Location	Rule
<code>SUPABASE_SERVICE_ROLE_KEY</code>	Server-only env var	Never in NEXT_PUBLIC_*; never in frontend code
<code>STRIPE_SECRET_KEY</code>	Server-only env var	Never in frontend; never in docs/examples
<code>STRIPE_WEBHOOK_SECRET</code>	Server-only env var	Never logged; never in responses
<code>VEKTRUM_TOKEN_SIGNING_KEY_PRIVATE</code>	Server-only env var	ed25519 private key PEM; never exposed
Partner API keys	Database (hashed)	Raw shown once; SHA-256 hash stored

SECTION 8 — PARTNER API DOCTRINE

8.1 Purpose

The Partner API is the integration surface for title companies, escrow companies, lenders, treasury teams, and institutional partners who execute payment externally after Vektrum authorizes a release, then confirm or fail execution through a machine-readable API.

8.2 Partner Capabilities

Action	Endpoint	Notes
Get release details	<code>GET /api/partner/releases/:id</code>	Partner-scoped; returns authorization, amount, execution status
Confirm external execution	<code>POST /api/partner/releases/:id/confirm</code>	Records method, reference, proof, partner_ack_hash
Fail external execution	<code>POST /api/partner/releases/:id/fail</code>	Records failure reason; triggers reservation cancellation
List pending releases	<code>GET /api/partner/releases?status=pending</code>	Partner-scope only
Introspect token	<code>GET /api/partner/tokens/:jti</code>	Status, scope, amounts, expiry; cross-partner blocked with 403

8.3 Partner Cannot

- Bypass the release gate
- Create releases directly
- Move funds through Vektrum
- Access deals not assigned to their partner_id
- Change billing rates, roles, deal terms, or audit records
- Call admin-only endpoints

8.4 Idempotency Requirements

Confirm and fail endpoints must be safe to call multiple times. A second confirm on an already-confirmed release returns 200 with `alreadyConfirmed: true`. A second fail returns 200 with `alreadyFailed: true`. Every partner confirmation must bind `partner_ack_hash` (SHA-256 of raw request body) and `token_hash` to the audit row.

8.5 Documentation Rules

Outbound webhook rule: Partner docs must NOT claim outbound webhooks are live unless implementation exists and has been tested. If not implemented, use: "Outbound partner webhooks are not required for the initial Partner API flow. Partners may poll `GET /api/partner/releases?status=pending`." Mark future webhook behavior as "Planned."

8.6 Safe Partner Messaging

Context	Approved Wording
What Vektrum does for partners	"Vektrum authorizes or blocks the release. The partner-controlled process executes payment."
Strategic value	"Keep your payment process. Add release enforcement."
Integration model	"Partners confirm execution after Vektrum authorizes release."
What Vektrum records	"Vektrum records method, reference, proof, actor, timestamp, and audit trail."

SECTION 9 — ADMIN DASHBOARD DOCTRINE

9.1 Required Dashboard Sections

Section	Route	Purpose
Overview	/dashboard/admin	Summary metrics; links to sections
Users	/dashboard/admin/users	List all users; link to user detail
User Detail	/dashboard/admin/users/[userId]	Name, email, role, Stripe status, deals
Partners / API Integrations	/dashboard/admin/partners	API key lifecycle; deal assignment
Audit Log	/dashboard/admin/audit	Read-only audit event viewer
Reconciliation	/dashboard/admin/reconciliation	Release reconciliation status

9.2 Admin Promotion Restrictions

Default state: ADMIN_PROMOTION_ENABLED=false

When false:

- No promote button exists in the dashboard
- /api/admin/promote returns 403 for all callers
- No way for any user to elevate to admin via the application

When true (intentional, operator-enabled):

- Caller must be admin role + MFA (AAL2) passed
- Self-promotion is blocked
- Rate limiting applies
- Admin justification required (X-Admin-Justification header, ≥ 20 chars)
- Full audit log entry written + dual write to admin_audit_log

9.3 UI Consistency Rules

Admin dashboard UI must use consistent dark background styling, uniform row heights in tables, uniform badge styles for status indicators, uniform button styles, consistent navigation (back links,

breadcrumbs, section labels), no light-theme modals appearing in dark-theme context, and no inconsistent error card styles.

SECTION 10 — DEMO SYSTEM DOCTRINE

10.1 Demo Reset Safety Rules

✓ DEMO RESET MUST

- Affect only demo data (demo deal IDs/slugs)
- Be idempotent (calling twice = same clean state)
- Require authentication (admin or demo-admin role)
- Be disabled in production unless `DEMO_RESET_ENABLED=true`
- Log the reset operation to the audit trail
- Return machine-readable success/failure response

✗ DEMO RESET MUST NOT

- Accept arbitrary user-provided deal IDs as targets
- Perform broad deletes without demo-scope filtering
- Expose service-role secrets to frontend
- Touch live production data under any circumstances
- Trust caller-provided IDs without demo scope verification

10.2 Demo Routes

Route	Purpose
<code>/demo-live</code>	Primary demo entry; directs to guided demo
<code>/demo-live/deal/harbor</code>	Harbor Draw #3 — primary demo scenario (\$2.18M)
<code>/demo-live/deal/riverside</code>	Secondary demo scenario
<code>/demo-live/admin</code>	Demo admin view (operational visibility demo)
<code>/demo-live/funder</code>	Funder dashboard demo

10.3 Demo Copy Rules

Approved: "This is a simulated demo. No real funds are moved." / "Authorization is demonstrated. Payment execution would flow through your selected rail."

Banned: "Vektrum moves money to the contractor" / "Project Trust Account" / "Vektrum Project Trust Agreement" / Specific AI provider stack names in public-facing demo copy.

SECTION 11 — COPYWRITING DOCTRINE

11.1 Approved Language

Concept	Approved Wording
What Vektrum is	Conditional authorization infrastructure for construction disbursements
Core capability	Release-control infrastructure; 10-condition release gate
Authorization principle	Authorization separated from execution
Payment language	Partner-controlled payment process; external/manual execution; Stripe Connect automated execution
Who governs	Vektrum governs authorization and records proof
Custody language	Vektrum does not hold funds in its own bank account or act as escrow
Audit trail	Append-only, hash-chained, tamper-evident audit log
AI pre-review	AI-assisted draw pre-review; AI informs; the gate decides
Gate promise	All required conditions must pass before a release is authorized
Trigger	Funder-triggered, system-enforced

11.2 Banned Language and Corrections

Banned Phrase	Approved Replacement
"Vektrum moves money"	"Vektrum authorizes releases; payment flows through your selected rail"
"Vektrum executes wires"	"Partners execute wires; Vektrum records authorization and proof"
"Vektrum holds funds"	"Funds remain with your existing custody infrastructure"
"Trust account" / "Project Trust Account"	Avoid entirely; remove from all copy
"Escrow replacement"	"Vektrum integrates with your existing escrow process"
"AI approves" / "AI clears"	"AI informs; the gate decides"
"Tamper-proof"	"Tamper-evident"
"Stripe required"	"Stripe Connect is one supported execution rail"
"7-condition gate" or "8-condition gate"	"10-condition release gate"
"Authorise" / "authorisation"	"Authorize" / "authorization" (US English throughout)

11.3 How It Works — Correct Structure

The "How It Works" page must be operational, not pitch-deck focused. Steps in correct sequence:

1. Draw submitted with supporting documentation
2. Documentation reviewed; AI-assisted draw pre-review flags risk signals
3. Release gate evaluates all required conditions deterministically
4. Funder authorizes release (all conditions passed)
5. Selected rail executes: Stripe Connect automated transfer or partner-controlled external execution
6. Audit trail updated: hash-chained event recorded with proof, method, reference, and actor
7. Reconciliation state updated: billing records, deal financials, token status confirmed

SECTION 12 — INVESTOR AND MARKET DOCTRINE

12.1 How to Explain to Investors

Opening framing: "Every construction loan requires a draw process. That draw process has near-zero software investment at the enforcement layer. Draw governance still runs on email, PDFs, and the judgment of individual underwriters. Vektrum places a deterministic, auditable enforcement layer between draw approval and fund disbursement."

Market size framing: \$550B+ annual US construction starts. Every one requires a draw process. The release-control layer has seen near-zero software investment. One SKU (release governance) attaches to every disbursement = high-frequency, high-margin SaaS on top of existing loan volume.

Network effect: Each lender brings multiple deals; each deal brings contractors already proven on the platform. Compounding switching cost.

Do not claim in investor conversations: AI company status, revenue figures you cannot substantiate, signed customers when only in design-partner conversations, Stripe-only rail, or "tamper-proof" audit.

12.2 How to Explain to Construction Lenders

Opening: "You have an existing draw process. Vektrum doesn't replace any of that. It sits before your disbursement execution and verifies that all required conditions are met before any dollar moves."

Key benefits: Reduces risk of fund misapplication. Creates evidence for LP reporting and regulatory compliance. Mechanic's lien protection via lien waiver enforcement. Reduces human judgment load on individual underwriters.

12.3 How to Explain to Title Companies

Opening: "Vektrum is not a title replacement. Your title company process stays exactly as it is. Vektrum is the layer the lender uses to verify release conditions are met before they authorize you to release funds."

Integration story: You receive a release authorization signal from Vektrum. You disburse through your existing infrastructure. You confirm back through the API. Both parties share a permanent audit record.

12.4 How to Explain to Contractors

Opening: "Vektrum shows you exactly what conditions are required for each draw to release and the current status of each condition. If your lien waiver is missing, you'll see it. If there's an open change order, you'll see it. No mystery about why a draw is delayed."

12.5 How to Explain to Technical Reviewers

- Next.js 15 application; Supabase (PostgreSQL + RLS) for persistent storage
- Row-Level Security on all 25+ tables
- Hash-chained, append-only audit log with SHA-256 row and chain hashes
- ed25519-signed authorization tokens per release (graceful fallback to unsigned)
- Rate limiting via database RPC (`check_rate_limit`)
- Stripe Connect for automated rail; Partner API for institutional rail
- AI draw review via sonar-pro (primary) → claude-sonnet (fallback) → gpt-4o (fallback)
- 100+ automated tests across release gate, security, webhook idempotency, admin safety, demo safety
- Migration-based schema management; immutability triggers on `authorization_tokens`

SECTION 13 — ENGINEERING DOCTRINE

13.1 Before Any Edit

1. Read `CLAUDE.md` at the repo root
2. Read `docs/ai/MASTER_CONTEXT.md` — current architecture and non-negotiables
3. Read `docs/ai/BACKLOG.md` — current priorities and what not to touch
4. Identify exactly which files are relevant — do not load unnecessary context
5. Produce a plan describing what will change and why
6. For CRITICAL or HIGH risk areas, wait for explicit approval before executing

13.2 After Any Edit

1. Run `npm run build` — document any new errors vs pre-existing
2. Run `npx tsc --noEmit` — zero new TypeScript errors
3. Run relevant tests: `npx tsx tests/[specific].test.ts`
4. Run `npm test` for full suite if shared infrastructure changed
5. Document exactly which files changed and why
6. Provide manual test steps for QA if UI changed
7. Confirm no secrets, real API keys, or real customer data are in the changeset

13.3 Test Expectations (TDD Required)

1. Write failing test describing expected behavior
2. Run test — confirm it fails for the right reason
3. Implement the smallest change that makes the test pass
4. Run all tests — confirm no regressions
5. Refactor if needed — tests protect behavior

13.4 Safe vs Dangerous Areas

Area	Risk Level	Rule
Release gate logic	CRITICAL	Do not modify without test coverage and explicit approval
Authorization token issuer	CRITICAL	Do not modify without understanding B3 hardening implications
<code>reserve_release_funds()</code> RPC	CRITICAL	Do not change behavior; double-spend protection
Audit chain hashing	HIGH	Do not change algorithm or payload structure without versioning
Admin promotion logic	HIGH	Default off; MFA required; self-promotion blocked
Billing calculation	HIGH	Do not change <code>calculateFee()</code> without test coverage
RLS policies	HIGH	Do not disable or weaken; test all changes
Partner API authentication	HIGH	Hash comparison; do not expose raw keys
Public copy	MEDIUM	Requires copy truth-lock review per Section 11
Demo reset	MEDIUM	Demo scope filter required
UI components (non-financial)	LOW-MEDIUM	Follow approved language
Test files	LOW	Adding tests is always encouraged

13.5 Rails Adapter Pattern — Do Not Bypass

```
// CORRECT: Use rail adapter
const adapter = getRailAdapter(railScope)
const dispatchResult = await adapter.dispatch({ ... })

// INCORRECT: Inline Stripe call in route – NOT ALLOWED
const transfer = await stripe.transfers.create({ ... })
```

13.6 Authorization Token Issuance — Required Fields (B3 Hardening)

```
await issueAuthorizationToken({
  milestoneId,
  dealId,
  payeeId,
  funderId,
  railScope,
  netToContractor,
  grossAmount,
  feeAmount,          // B3 hardening — REQUIRED. Do not omit.
  idempotencyKey,
  issuedBy,
  graphCommitment, // Tier D — if evidence graph is built
})
```

SECTION 14 — RELEASE CHECKLIST

14.1 Build and Test

- ☐ `npm run build` passes with no new errors
- ☐ `npx tsc --noEmit` — zero TypeScript errors
- ☐ `npm test` — all tests pass or pre-existing failures are documented
- ☐ New tests written for all behavior changes (TDD)
- ☐ No new `console.error` or unhandled promise rejections in build output

14.2 Security

- ☐ No real API keys, secrets, or tokens in the changeset
- ☐ No `SUPABASE_SERVICE_ROLE_KEY` in `NEXT_PUBLIC_*` variables
- ☐ Admin promotion remains disabled by default
- ☐ Admins cannot trigger release via any new routes
- ☐ Partners cannot bypass the gate via any new routes
- ☐ New financial-write routes have rate limiting applied
- ☐ New admin routes require MFA
- ☐ RLS is still enabled on all tables touched by migrations
- ☐ Stripe webhook handling includes HMAC verification and event deduplication

14.3 Copy and Legal-Risk

- ☐ No "Vektrum moves money" or equivalent
- ☐ No "Vektrum holds funds" or equivalent
- ☐ No "escrow replacement" or "trust account" or "Project Trust Account"
- ☐ No "AI approves" or "AI decides"
- ☐ No "tamper-proof" — only "tamper-evident"
- ☐ No "Stripe required" — external rail messaging present where relevant
- ☐ Gate described as "10-condition" — not 7 or 8
- ☐ Partner docs do not overclaim outbound webhooks if unimplemented
- ☐ Demo copy does not claim real fund movement

14.4 Partner API

- ☐ No new partner endpoints that bypass the release gate
- ☐ Partner authentication remains hash-based
- ☐ Partner scope isolation enforced (cross-partner access returns 403)
- ☐ Confirm and fail endpoints are idempotent
- ☐ Audit rows bind partner_ack_hash and token_hash
- ☐ Postman collection uses placeholder keys only

SECTION 15 — AI AGENT BEHAVIOR RULES

15.1 Reasoning Order

1. Read this document first. No work begins without it.
2. Identify the task category: code, copy, docs, security, or architecture.
3. Check this document for governing rules on that category.
4. Identify the minimum required change that achieves the goal.
5. Check for dangerous areas (Section 13.4). If CRITICAL or HIGH, state this explicitly.
6. Produce a plan before executing. For CRITICAL areas, wait for explicit approval.
7. Execute with precision. No scope creep. State any "while I'm here" changes explicitly.
8. Verify with tests and build. Do not claim success without running the checks.
9. Report honestly: if tests fail, say so; if something was skipped, say so.

15.2 When to Stop and Ask

The AI must stop and ask for explicit approval before proceeding when the task involves:

- Any change to the release gate logic
- Any change to billing or financial calculations
- Any change to admin security boundaries
- Any change to audit log behavior or hashing scheme
- Any copy change that makes custody, escrow, or AI approval claims
- Any new route that could create a path for non-funder actors to trigger release
- Any test fixture that might touch production data

15.3 Required Change Summary Format

```
## Change Summary

Files modified:
  - [file path]: [one-sentence description of what changed and why]

Tests added or modified:
  - [test file]: [what behavior is now tested]

Build/test status:
  - npm run build: [pass/fail]
  - npx tsc --noEmit: [pass/fail / N new errors]
  - npm test: [pass/fail] [count of failing tests if any]
  - Pre-existing errors: [list if any]

Security/custody/gate risk: [low / medium / high] – [reason]
Copy/legal risk: [low / medium / high] – [reason]

Manual test steps:
  - [step 1]
  - [step 2]

Next recommended action: [specific concrete next step]
```

15.4 What the AI Must Never Do Unilaterally

Without explicit approval, the AI must never:

- Change the 10 release conditions or their labels
- Change the billing calculation formula
- Change admin role permissions
- Change partner scope isolation logic
- Disable rate limiting on any route
- Weaken RLS policies
- Add any pathway for admins to release funds
- Change the authorization token hashing scheme
- Change the audit chain hash algorithm or payload structure
- Remove MFA requirements from any route
- Add public copy that makes custody, escrow, or AI approval claims
- Commit real API keys, secrets, or tokens

SECTION 16 — EXAMPLE RESPONSES AND PATTERNS

16.1 To a Title Company

"Not at all. Vektrum is the governance layer that your lender client uses to verify that all release conditions are met before they authorize you to release funds. Your disbursement process stays exactly as it is. We integrate with your process through our Partner API — when you release funds on behalf of a lender using Vektrum, you confirm the execution through our API and both parties get a shared audit record of what was authorized, when, and on what basis."

16.2 To a Construction Lender

"Vektrum sits before your title and escrow process, not instead of it. When a draw is submitted, Vektrum evaluates all required conditions — lien waivers, change orders, contract status, funding availability, contractor onboarding, and AI-assisted draw review. If everything passes, you authorize the release and Vektrum records that authorization with a full audit trail. Your LP reporting now has a machine-readable trail of every release: what was authorized, why, by whom, and what documentation was on file."

16.3 To an Investor

"A general payments company solves 'how do we move money faster.' Vektrum solves 'how do we verify that money is allowed to move at all, and how do we prove it?' That's a fundamentally different problem. And because Vektrum works with whatever payment rail the lender already uses — Stripe Connect for automated releases, or their existing title/escrow/treasury process — there's no switching cost. We're not replacing the payment rail. We're enforcing whether the payment should happen at all."

16.4 Correcting Banned Wording

Draft (Banned)	Corrected (Approved)
"Vektrum's AI clears your draws for payment"	"Vektrum's AI-assisted draw pre-review flags missing documentation, conflicts, and risk signals. The deterministic gate then evaluates all required conditions. AI informs; the gate decides."
"Funds are held in Vektrum's trust account"	"Funds remain with your existing custody infrastructure; Vektrum enforces release conditions before authorizing disbursement"
"Tamper-proof audit trail"	"Tamper-evident, hash-chained, append-only audit trail"
"8 required conditions"	"10 required conditions"
"Escrow replacement for modern lenders"	"Release governance for construction lenders, integrating with your existing title, escrow, and treasury processes"
"Our AI approves draws in seconds"	"AI-assisted draw review runs before the deterministic gate — flagging missing documents, conflicts, and risk signals"

SECTION 17 — CURRENT KNOWN BACKLOG

17.1 Critical / Before Merge

Item	Status	Action Required
<code>/auth/logout</code> 404	Needs verification	Verify logout route exists; fix if returning 404
Dashboard user navigation	Partly addressed	Verify <code>/dashboard/admin/users/[userId]</code> loads with user details and deal links
Partner API docs webhook claims	Needs verification	Audit docs; replace live webhook claims with "Planned" if unimplemented

17.2 High / Before Pilot

Item	Acceptance Criteria
Route/navigation smoke tests	All critical routes verified non-404 via test or checklist
Admin dashboard full verification	No promote button; admin promotion disabled; partners section works; no raw secrets exposed
Demo reset full verification	Demo-scope only; idempotent; no production data touched; clean reload after reset

17.3 Test Priorities

1. Release gate tests — all 10 conditions + bypass attempts
2. Partner API confirm/fail tests — idempotency, scoping, audit binding
3. Admin promote disabled tests — verify default-off behavior
4. Demo reset safety tests — verify demo-scope isolation
5. Route existence/navigation smoke tests
6. Stripe webhook idempotency tests
7. RLS/DB trigger expectation tests

17.4 Do Not Touch Without Explicit Instruction

- Release gate logic (`src/lib/engine/release-gate.ts`)
- Billing calculation formula (`src/lib/engine/billing.ts`)
- Stripe webhook transaction logic
- Admin permission boundaries
- Partner API request/response shapes
- Audit chain hashing scheme
- RLS policies
- Authorization token immutability triggers
- Public site copy, unless product behavior has changed

SECTION 18 — FINAL NON-NEGOTIABLES

These are the most important rules in this document. They must be honored in every piece of work, every AI output, every line of code, and every word of copy. There are no exceptions.

1 Vektrum does not hold funds in its own bank account or act as escrow.

Funds are held by Stripe Connect (for Stripe rail deals) or by the funder's existing title company, escrow company, bank, or treasury infrastructure. Any copy, code, or AI output that implies otherwise must be corrected.

2 Authorization is separated from execution.

Vektrum authorizes whether a release is allowed. Payment execution happens through the selected rail — not through Vektrum. The rail adapter exists to enforce this separation in the codebase.

3 Admins cannot release funds.

The admin role is operational. No dashboard feature, no API endpoint, no code path should give admin users the ability to trigger a construction release. Any such path is a critical security bug.

4 Partners cannot bypass the release gate.

Partner API endpoints confirm or fail external execution after authorization. They do not create authorization, and they do not skip the gate. Any partner endpoint that creates a release without gate evaluation is a critical bug.

5 AI informs; the deterministic gate decides.

No work should result in copy, code, or documentation that implies AI makes the release decision. The AI pre-review runs before the gate. The gate — all 10 conditions — decides.

6 Do not overclaim webhooks, custody, automation, or payment execution.

Do not document outbound webhooks as live unless the implementation exists. Do not claim Stripe is required. Do not claim Vektrum automates all payments. Do not claim Vektrum is an escrow or custody provider.

7 Public copy must stay rail-neutral and legally careful.

The public site must not imply Stripe is the only rail, use trust account language, imply Vektrum holds funds, use "tamper-proof," describe AI as approving payments, or claim a gate condition count other than 10.

8 The 10-condition gate is fixed and public.

The gate has exactly 10 conditions. They must not be described as 7, 8, or 9. They must not be reordered, removed, or combined without explicit product approval. Any test, copy, or documentation describing fewer conditions is incorrect.

9 Raw API keys are shown once. Never again.

The Partner API key is generated, shown once at creation, and stored only as a SHA-256 hash. If a partner loses their key, it must be rotated. Any UI that shows a raw key after initial creation is a security bug.

10 No service-role secrets in the frontend. Ever.

`SUPABASE_SERVICE_ROLE_KEY` , `STRIPE_SECRET_KEY` , `VEKTRUM_TOKEN_SIGNING_KEY_PRIVATE` , and all equivalent server-side secrets must never appear in `NEXT_PUBLIC_*` environment variables, client components, or any code bundled into the browser.